

Resumo Teórico — Paradigmas de Programação (PP)

Guia de Estudo para Exame de Recurso | Ano Letivo: 2025/2026

ESTG — P.PORTO | LEI / LSIRC

1. Tipos Primitivos, Variáveis e Arrays

1.1 Tipos Primitivos

Java possui 8 tipos primitivos. Os tipos primitivos armazenam o valor diretamente na Stack e não são objetos.

`byte` (8 bits), `short` (16 bits), `int` (32 bits), `long` (64 bits), `float` (32 bits), `double` (64 bits), `char` (16 bits Unicode), `boolean` (true/false).

Os valores por defeito dos tipos primitivos quando declarados como atributos de instância são: 0 para numéricos, `'\u0000'` para char e `false` para boolean. As variáveis locais (dentro de métodos) não possuem valores por defeito e o compilador obriga à sua inicialização antes de serem utilizadas.

1.2 Constantes

As constantes em Java são declaradas com a palavra reservada `final`. Uma vez atribuído um valor, este não pode ser alterado. Por convenção, os nomes de constantes são escritos em maiúsculas com underscores.

```
final double PI = 3.14159;  
final int MAX_ELEMENTOS = 100;
```

1.3 Arrays

Os arrays em Java são estruturas de dados de tamanho fixo que armazenam elementos do mesmo tipo. O tamanho é definido no momento da criação e não pode ser alterado posteriormente. Os arrays de tipos primitivos são inicializados com os valores por defeito (0, false, etc.). Os arrays de objetos são inicializados com `null` em todas as posições.

```
int[] numeros = new int[5];           // Array de primitivos (tudo a 0)
String[] nomes = new String[3];       // Array de objetos (tudo a null)
int[] preDef = {1, 2, 3, 4, 5};       // Inicialização direta
```

Para "redimensionar" um array é necessário criar um novo array e copiar os elementos manualmente, pois os arrays na Heap são blocos contíguos de tamanho imutável.

2. Classes e Objetos

2.1 Conceito de Classe

Uma classe é um molde ou modelo que define a estrutura (atributos) e o comportamento (métodos) que os objetos criados a partir dela terão. Uma classe é composta por atributos (variáveis de instância que representam o estado) e métodos (funções que definem as operações sobre o estado).

```
public class Contendor {
    private String codigo;           // Atributo de instância
    private double capacidade;       // Atributo de instância

    public String getCodigo() {      // Método de instância
        return codigo;
    }
}
```

2.2 Criação de Objetos

Os objetos são criados utilizando o operador `new`, que aloca memória na Heap e invoca o construtor da classe. A variável que armazena o objeto contém apenas uma referência (endereço de memória) para o espaço alocado na Heap.

```
Contendor c = new Contendor();
// 'c' é uma referência na Stack que aponta para o objeto na Heap
```

2.3 Variáveis de Instância vs Variáveis de Classe (static)

As variáveis de instância pertencem a cada objeto individual, existindo uma cópia por objeto. As variáveis de classe (`static`) pertencem à classe e são compartilhadas por todas as instâncias, existindo apenas uma cópia na memória.

```
public class Veiculo {  
    private String matricula;           // Variável de instância (uma por objeto)  
    private static int totalVeiculos = 0; // Variável de classe (partilhada)  
  
    public Veiculo(String matricula) {  
        this.matricula = matricula;  
        totalVeiculos++; // Incrementa o contador global  
    }  
  
    public static int getTotalVeiculos() { // Método estático  
        return totalVeiculos;  
    }  
}
```

Os métodos estáticos não podem aceder a variáveis de instância nem usar `this`, porque não existe nenhum objeto de contexto associado.

3. Construtores

3.1 Definição

Os construtores são blocos especiais que são invocados automaticamente ao criar um objeto com `new`. Têm obrigatoriamente o mesmo nome da classe e não possuem tipo de retorno (nem `void`). A sua função é inicializar o estado do objeto.

3.2 Construtor por Defeito

Quando uma classe não define nenhum construtor, o compilador gera automaticamente um construtor por defeito (sem parâmetros) que invoca `super()` e inicializa os atributos com valores padrão. Se a classe definir pelo menos um construtor explícito, o compilador deixa de gerar o construtor por defeito.

3.3 Encadeamento de Construtores

A palavra `this(...)` permite invocar outro construtor da mesma classe, evitando duplicação de código. A palavra `super(...)` permite invocar um construtor da superclasse. Ambos devem ser a primeira instrução do construtor e são mutuamente exclusivos.

```
public class AidBox {  
    private String code;  
    private String zone;  
  
    public AidBox(String code) {  
        this(code, "Desconhecida");    // Invoca o outro construtor  
    }  
  
    public AidBox(String code, String zone) {  
        this.code = code;  
        this.zone = zone;  
    }  
}
```

4. Encapsulamento e Modificadores de Acesso

4.1 Conceito de Encapsulamento

O encapsulamento consiste em ocultar os detalhes internos de implementação de uma classe, expondo apenas uma interface pública controlada. Os atributos devem ser declarados como `private` e o acesso ao estado deve ser feito exclusivamente através de métodos getters e setters que podem incorporar lógica de validação.

4.2 Modificadores de Acesso

`private` — Acesso apenas dentro da própria classe. É o mais restritivo.

`protected` — Acesso dentro da própria classe, no mesmo pacote e em subclasses (mesmo que noutro pacote).

Sem modificador (package-private) — Acesso dentro do mesmo pacote apenas.

`public` — Acesso universal a partir de qualquer classe em qualquer pacote.

4.3 Getters e Setters com Validação

```
public class Container {  
    private double capacidade;  
    private double pesoAtual;  
  
    public double getCapacidade() {  
        return capacidade;  
    }  
  
    public void setPesoAtual(double peso) {  
        if (peso < 0 || peso > capacidade) {  
            throw new IllegalArgumentException("Peso invalido.");  
        }  
        this.pesoAtual = peso;  
    }  
}
```

5. Enumerações (Enum)

Os tipos enumerados (`enum`) representam conjuntos fixos e predefinidos de constantes. Cada valor é uma instância singleton. A comparação entre valores de enum pode ser feita com `==` de forma segura. Os enums podem possuir atributos, construtores (obrigatoriamente privados) e métodos.

```
public enum ItemType {  
    PERISHABLE_FOOD("Alimentos Pereciveis"),  
    NON_PERISHABLE_FOOD("Alimentos Nao Pereciveis"),  
    CLOTHING("Vestuario"),  
    MEDICINE("Medicamentos");  
  
    private String descricao;  
  
    ItemType(String descricao) {  
        this.descricao = descricao;  
    }  
  
    public String getDescricao() {  
        return descricao;  
    }  
}
```

Vantagens sobre constantes inteiras ou Strings: segurança de tipos em tempo de compilação, legibilidade, suporte nativo para `switch`, e comparação segura com `==`.

6. Métodos e Sobrecarga

6.1 Definição de Métodos

Um método é um bloco de código que executa uma tarefa específica. É definido pela sua assinatura (nome + lista de parâmetros) e pelo tipo de retorno.

6.2 Sobrecarga de Métodos (Overloading)

A sobrecarga ocorre quando uma classe define vários métodos com o mesmo nome mas assinaturas diferentes (diferente número e/ou tipos de parâmetros). A resolução é feita em tempo de compilação.

```
public int somar(int a, int b) { return a + b; }  
public double somar(double a, double b) { return a + b; }  
public int somar(int a, int b, int c) { return a + b + c; }
```

7. Passagem de Argumentos

Em Java, todos os argumentos são passados por valor (pass-by-value).

Para tipos primitivos, é passada uma cópia do valor. Alterações dentro do método não afetam a variável original.

Para tipos de referência (objetos), é passada uma cópia da referência (endereço de memória). Como ambas as referências apontam para o mesmo objeto na Heap, alterações ao estado interno do objeto são visíveis fora do método. Contudo, reatribuir a referência dentro do método (`param = new Objeto()`) não afeta a referência original, porque apenas a cópia local é alterada.

```
public static void alterarEstado(int[] arr) {  
    arr[0] = 999;    // Modifica o objeto real na Heap – visível fora  
}  
  
public static void reatribuir(int[] arr) {  
    arr = new int[]{1, 2, 3};    // Só altera a cópia local da referência  
}
```

8. Herança

8.1 Conceito

A herança permite que uma subclasse herde os atributos e métodos de uma superclasse, promovendo a reutilização de código. Em Java, a herança é simples: uma classe só pode estender uma única superclasse. Expressa-se com a palavra reservada `extends`.

8.2 A Palavra Reservada `super`

No construtor, `super(...)` invoca o construtor da superclasse e deve ser a primeira instrução. Em métodos, `super.nomeDoMetodo()` invoca a versão do método da superclasse que foi sobreposta.

```
public class Veiculo {  
    private String matricula;  
  
    public Veiculo(String matricula) {  
        this.matricula = matricula;  
    }  
}  
  
public class Camiao extends Veiculo {  
    private double cargaMax;  
  
    public Camiao(String matricula, double cargaMax) {  
        super(matricula);        // Invoca construtor de Veiculo  
        this.cargaMax = cargaMax;  
    }  
}
```

8.3 Sobreposição de Métodos (Overriding)

A sobreposição ocorre quando uma subclasse redefine um método herdado mantendo a mesma assinatura. A anotação `@Override` é recomendada para validação pelo compilador. A resolução é feita em tempo de execução pela JVM (ligação dinâmica / dynamic binding).

8.4 A Classe Object

Todas as classes em Java estendem implicitamente `Object`. Métodos herdados que devem ser considerados para redefinição: `equals(Object obj)`, `toString()`, `getClass()`.

9. Casting (Conversão de Tipos)

9.1 Upcasting (Conversão Ascendente)

Conversão de subclasse para superclasse. É implícita e sempre segura. Perde-se acesso aos métodos específicos da subclasse, mas o polimorfismo continua a funcionar.

```
Vehicle v = new RefrigeratedVehicleImpl("V-001", ...); // Upcasting
v.getCode(); // OK
// v.getMaxKilometers(); // ERRO de compilação
```

9.2 Downcasting (Conversão Descendente)

Conversão de superclasse para subclasse. É explícita e potencialmente perigosa. Se o objeto real não for do tipo pretendido, lança `ClassCastException` em runtime. Deve-se usar `instanceof` antes do cast.

```
if (v instanceof RefrigeratedVehicle) {
    RefrigeratedVehicle rv = (RefrigeratedVehicle) v; // Downcasting seguro
    rv.getMaxKilometers();
}
```

10. Classes Abstratas

Uma classe abstrata é uma classe que não pode ser instanciada diretamente e pode conter métodos abstratos (sem implementação) e métodos concretos (com implementação). Declara-se com a palavra `abstract`. As subclasses concretas são obrigadas a implementar todos os métodos abstratos.


```
public abstract class Transporte {  
    private String id;  
  
    public Transporte(String id) {  
        this.id = id;  
    }  
  
    public String getId() { return id; } // Método concreto  
    public abstract double calcularCusto(double km); // Método abstrato  
}  
  
public class Camiao extends Transporte {  
    private double custoPorKm;  
  
    public Camiao(String id, double custoPorKm) {  
        super(id);  
        this.custoPorKm = custoPorKm;  
    }  
  
    @Override  
    public double calcularCusto(double km) {  
        return km * custoPorKm;  
    }  
}
```

Usada para representar uma relação "É UM" com partilha de estado e comportamento comum. O modificador `final` impede que uma classe seja estendida, que um método seja sobreposto ou que um atributo seja reatribuído.

11. Interfaces

Uma interface define um contrato de comportamento que as classes podem implementar. Declara-se com `interface` e implementa-se com `implements`. Uma classe pode implementar múltiplas interfaces (herança múltipla de tipo).

```
public interface Vehicle {  
    String getCode();  
    ItemType getSupplyType();  
    double getMaxCapacity();  
}  
  
public interface RefrigeratedVehicle extends Vehicle {  
    double getMaxKilometers();  
}  
  
public class RefrigeratedVehicleImpl implements RefrigeratedVehicle {  
    // Deve implementar TODOS os métodos de Vehicle e RefrigeratedVehicle  
}
```

Desde o Java 8, interfaces podem ter métodos `default` (com implementação padrão) e métodos `static`. Desde o Java 9, podem ter métodos `private`.

12. Classes Abstratas vs Interfaces — Comparação Direta

Classes Abstratas: Herança simples. Podem ter atributos de instância mutáveis. Podem ter construtores. Podem ter métodos concretos e abstratos. Relação de identidade ("É UM").

Interfaces: Herança múltipla de tipo. Apenas constantes (`public static final`). Sem construtores. Métodos abstratos, default e static. Contrato de comportamento ("CONSEGUE FAZER").

Usar classe abstrata quando se partilha estado e código entre classes da mesma família. Usar interface quando se define um contrato para classes de famílias diferentes.

13. Polimorfismo

13.1 Polimorfismo de Sobreposição (Runtime)

A JVM determina em tempo de execução qual a versão do método a invocar, consultando a classe real do objeto na Heap (dynamic binding). Permite tratar objetos de subclasses de forma uniforme através de referências da superclasse.

```
Vehicle v = new RefrigeratedVehicleImpl(...);  
v.getCode();    // Executa a versão de RefrigeratedVehicleImpl
```

13.2 Polimorfismo de Sobrecarga (Compile-time)

O compilador escolhe qual dos métodos sobrecarregados invocar com base nos tipos dos argumentos na chamada.

14. Identidade vs Igualdade de Objetos

14.1 Operador `==`

Para tipos primitivos, compara valores. Para objetos, compara referências (endereços de memória): verifica se duas variáveis apontam para o mesmo objeto.

14.2 Método `equals()`

Herdado de `Object`, por defeito compara referências (igual a `==`). Deve ser redefinido para comparar conteúdo lógico.

14.3 Estrutura Correta do `equals()`

```
@Override
public boolean equals(Object obj) {
    if (this == obj) return true;           // 1. Mesma referência
    if (obj == null) return false;          // 2. Nulo
    if (!(obj instanceof Vehicle)) return false; // 3. Tipo compatível
    Vehicle other = (Vehicle) obj;          // 4. Cast seguro
    return this.code.equals(other.getCode()); // 5. Comparação lógica
}
```

14.4 `instanceof` VS `getClass()` no `equals`

`instanceof` aceita subclasses e implementações de interface (mais flexível). `getClass()` exige exatamente a mesma classe (mais restritivo, garante simetria). Usar `instanceof` quando a igualdade é definida a nível de interface. Usar `getClass()` quando a igualdade é específica da classe concreta.

14.5 Método `toString()`

Por defeito retorna `NomeClasse@hashHex`. Deve ser redefinido para representação legível do estado. É invocado automaticamente por `System.out.println()`.

15. Exceções

15.1 Hierarquia

`Throwable` → `Error` (falhas graves do sistema, não tratáveis) e `Exception` (condições recuperáveis).
`Exception` → `RuntimeException` (não verificadas) e restantes (verificadas).

15.2 Checked vs Unchecked

Exceções verificadas (checked): subclasses de `Exception` que não são `RuntimeException`. O compilador obriga ao tratamento com `try-catch` ou à declaração com `throws`. Representam situações previsíveis.

Exceções não verificadas (unchecked): subclasses de `RuntimeException`. Não é obrigatório tratá-las. Representam erros de programação (`NullPointerException`, `ArrayIndexOutOfBoundsException`).

15.3 Mecanismo try-catch-finally

```
try {  
    route.addAidBox(aidBox);          // Código que pode lançar exceção  
} catch (RouteException e) {  
    System.out.println(e.getMessage()); // Tratamento da exceção  
} finally {  
    System.out.println("Executado sempre."); // Limpeza/finalização  
}
```

15.4 Criação de Exceções Personalizadas

```
public class RouteException extends Exception {  
    public RouteException(String message) {  
        super(message);  
    }  
}
```

Para lançar: `throw new RouteException("Mensagem de erro.");`

Para declarar que um método pode lançar: `public void addAidBox(AidBox ab) throws RouteException { ... }`

16. Composição vs Herança

Herança: Relação "É UM" (is-a). Acoplamento forte. A subclasse herda a implementação da superclasse. Alterações na superclasse podem afetar subclasses.

Composição: Relação "TEM UM" (has-a). Acoplamento fraco. Um objeto contém outro como atributo. Maior flexibilidade e independência entre classes.

```
// COMPOSIÇÃO: Veiculo TEM UM Motor
public class Veiculo {
    private Motor motor;    // Relação has-a

    public Veiculo(Motor motor) {
        this.motor = motor;
    }

    public void arrancar() {
        motor.ligar();
    }
}
```

A composição é frequentemente preferida porque permite substituir componentes em tempo de execução, reduz o acoplamento e evita a fragilidade da herança.

17. Membros Estáticos (static)

Atributos `static` pertencem à classe (uma única cópia partilhada). Métodos `static` podem ser invocados sem criar instâncias (`NomeClasse.metodo()`). Métodos `static` não podem aceder a `this` nem a membros de instância. Caso de uso clássico: contadores de instâncias, constantes de classe, métodos utilitários.

18. A Palavra `final`

Aplicada a variáveis: impede a reatribuição do valor (constante).

Aplicada a métodos: impede a sobreposição em subclasses.

Aplicada a classes: impede que a classe seja estendida (sem subclasses).

```
public final class Utilitarios { ... }    // Não pode ser estendida
public final double PI = 3.14159;        // Valor imutável
public final void metodo() { ... }       // Não pode ser sobreposto
```

19. Resumo de Palavras Reservadas Essenciais

`class` — Define uma classe. `new` — Cria um novo objeto (aloca na Heap). `this` — Referência para o objeto corrente. `super` — Referência para a superclasse. `extends` — Herda de uma classe. `implements` — Implementa uma interface. `abstract` — Classe ou método abstrato. `interface` — Define uma interface. `static` — Membro pertence à classe. `final` — Constante / não redefinível / não extensível. `private`, `protected`, `public` — Modificadores de acesso. `void` — Sem tipo de retorno. `return` — Devolve um valor. `throw` — Lança uma exceção. `throws` — Declara exceções no método. `try`, `catch`, `finally` — Tratamento de exceções. `instanceof` — Verifica tipo do objeto. `enum` — Tipo enumerado. `@Override` — Anotação de sobreposição.

20. Padrões de Código Frequentes em Exame

20.1 Implementar uma Interface com Estado

```
public class XImpl implements X {  
    private String code;  
    private TipoEstado state;  
  
    public XImpl(String code) {  
        this.code = code;  
        this.state = TipoEstado.VALOR_DEFEITO;  
    }  
  
    @Override  
    public String getCode() { return this.code; }  
  
    @Override  
    public boolean equals(Object obj) {  
        if (this == obj) return true;  
        if (obj == null) return false;  
        if (!(obj instanceof X)) return false;  
        X other = (X) obj;  
        return this.code.equals(other.getCode());  
    }  
  
    @Override  
    public String toString() {  
        return "X [" + code + " | " + state + "];"  
    }  
}
```

20.2 Gerir um Array Interno (Adicionar/Remover)

```
private Elemento[] elementos;
private int count;

public void add(Elemento e) throws MinhaExcecao {
    if (e == null) throw new MinhaExcecao("Nulo.");
    if (count >= elementos.length) throw new MinhaExcecao("Cheio.");
    // Verificar duplicados
    for (int i = 0; i < count; i++) {
        if (elementos[i].equals(e)) throw new MinhaExcecao("Duplicado.");
    }
    elementos[count] = e;
    count++;
}

public Elemento remove(Elemento e) throws MinhaExcecao {
    for (int i = 0; i < count; i++) {
        if (elementos[i].equals(e)) {
            Elemento removed = elementos[i];
            for (int j = i; j < count - 1; j++) {
                elementos[j] = elementos[j + 1];
            }
            elementos[count - 1] = null;
            count--;
            return removed;
        }
    }
    throw new MinhaExcecao("Nao encontrado.");
}
```

20.3 Devolver Array sem Nulos (Shrinking)

```
public Elemento[] getElementos() {
    Elemento[] result = new Elemento[count];
    for (int i = 0; i < count; i++) {
        result[i] = elementos[i];
    }
    return result;
}
```


20.4 Iterar e Filtrar com Condição

```
Elemento[] temp = new Elemento[total.length];
int filteredCount = 0;
for (int i = 0; i < total.length; i++) {
    if (total[i] != null && condicao(total[i])) {
        temp[filteredCount] = total[i];
        filteredCount++;
    }
}
Elemento[] result = new Elemento[filteredCount];
for (int i = 0; i < filteredCount; i++) {
    result[i] = temp[i];
}
return result;
```